

# Executable Unpacker: Windows PE Yürütülebilir Dosyaları için Hata Ayıklayıcı Tabanlı Dinamik Bellek Dökümü Aracı

Öğrenci Adı Soyadı

Bilgisayar Mühendisliği Bölümü  
Üniversite Adı — Ankara, Türkiye

**Özet**—Bu çalışmada, Windows işletim sistemi üzerinde çalışan paketlenmiş (packed) PE (Portable Executable) yürütülebilir dosyalarını, çalışma zamanında bellekten dökerek orijinal hâline geri döndürmek amacıyla geliştirilmiş bir araç sunulmaktadır. Araç; PEParse (statik analiz), DebuggerEngine (dinamik OEP tespiti), Dumper (bellek

dökümü) ve IATRebuilder (ithalat tablosu yeniden inşası) olmak üzere dört ana bileşenden oluşmaktadır. C++20 standardında ve SOLID tasarım ilkeleri gözetilerek gerçekleştirilen implementasyon; hem konsol (CLI) hem de yerel Win32 GUI arayüzüyle kullanılabilir. Donanım kesme noktası (Hardware Breakpoint) ve PAGE\GUARD mekanizmaları aracılığıyla Özgün Giriş Noktası (OEP) dinamik olarak tespit edilmekte; ardından bellek bölümleri disk üzerine yazılarak çalışabilir bir döküm dosyası elde edilmektedir.

**Anahtar Kelimeler**—PE dosyası, unpacker, hata ayıklama, OEP tespiti, IAT yeniden inşası, Windows güvenliği, C++20, nesne yönelimli programlama

Sistem; tek sorumluluk ilkesi (SRP) ve bağımlılığın tersine çevrilmesi (DIP) ilkeleri doğrultusunda dört bağımsız bileşene ayrılmıştır. Bileşenler arasındaki iletişim soyut arayüzler üzerinden sağlanmakta, somut sınıflar doğrudan birbirine bağlanmamaktadır.

Tablo I. İşlem Boru Hattı Aşamaları

| Evre | Bileşen        | Görev                            |
|------|----------------|----------------------------------|
| 1    | PEParser       | Statik PE başlık analizi         |
| 2    | DebuggerEngine | Süreç başlatma, OEP tespiti      |
| 3    | Dumper         | Bellekten bölüm okuma & kaydetme |
| 4    | IATRebuilder   | İthalat tablosu yeniden inşası   |
| 5    | Dumper.SaveAs  | Döküm dosyasını diske yazma      |

## I. GİRİŞ

Yazılım paketleyicileri (packer), kötü amaçlı yazılımların ve ticari uygulamaların tersine mühendislik analizini zorlaştırmak için PE dosyalarını sıkıştırır veya şifreler. Paketlenmiş bir yürütülebilir, çalışma zamanında kendini belleğe açar ve asıl koda (OEP — Original Entry Point) sıçar. Bu mekanizma statik analizörler için ciddi bir engel oluşturur.

Bu proje kapsamında geliştirilen **Executable Unpacker**, Windows Hata Ayıklama API'sini kullanarak hedef süreci çalıştırır, OEP anını yakalar ve o andaki bellek görüntüsünü geçerli bir PE dosyası olarak diske yazar. Araç; güvenlik araştırmaları, kötü amaçlı yazılım analizi ve tersine mühendislik eğitimi gibi savunmacı amaçlarla kullanılmak üzere tasarlanmıştır.

Raporun geri kalanında Bölüm II sistem mimarisi ve tasarım kararlarını, Bölüm III her bileşenin teknik detaylarını, Bölüm IV GUI implementasyonunu, Bölüm V derleme ve kullanım bilgilerini ve Bölüm VI sonuçları içermektedir.

## II. SİSTEM MİMARİSİ VE TASARIM KARARLARI

### A. İzlenen Tasarım Kalıpları

**Strategy Pattern:** *IUnpacker* soyut arayüzü, farklı OEP tespit stratejilerinin (hata ayıklayıcı, emülatör, korumalı alan vb.) aynı boru hattında kullanılabilmesini sağlar. Mevcut uygulama bu arayüzü *DebuggerEngine* ile somutlaştırmaktadır.

**Dependency Inversion Principle (DIP):** *Logger* sınıfı, çıkış hedefini bir *Sink* fonksiyon nesnesi aracılığıyla soyutlar. Konsol uygulaması *std::cout*'u, GUI uygulaması ise Win32 mesaj kuyruğunu *sink* olarak atar. Bu sayede motor kodu arayüze bağımlı değildir.

**RAII:** Tüm Win32 HANDLE kaynakları (süreç, iş parçacığı, dosya tanıtıcıları) yıkıcı metotlar aracılığıyla otomatik serbest bırakılır; bellek sızıntısı riski minimuma indirilir.

**Thread-Safe Singleton Türevi:** *Logger::Mutex()*, *inline static* depolama sınıfı sayesinde program ömrü boyunca tek bir mutex örneğine sahip olur ve birden fazla iş parçacığının eş zamanlı yazma sorununu önler.

## III. BİLEŞEN TEKNİK DETAYLARI

## A. IUnpacker Arayüzü

*IUnpacker.h* dosyasında tanımlanan bu soyut arayüz, üç saf sanal metot içermektedir: **Run()** (bloklayan ana çalışma döngüsü), **IsOEPFound()** ve **GetOEP()**. Gelecekte farklı unpack stratejileri eklendiğinde bu arayüz üzerinden çalışma sürecinde değiştirilebilir.

## B. DebuggerEngine

Windows Hata Ayıklama API'sinin merkezinde yer alan bu sınıf, hedef süreci *CREATE\_SUSPENDED* bayrağıyla başlatır, *DebugActiveProcess()* ile hata ayıklayıcı olarak bağlanır ve *WaitForDebugEvent* / *ContinueDebugEvent* çiftini döngüsel olarak çağırır.

### 1) Donanım Kesme Noktaları (DR0-DR3)

Intel SDM Vol.3 §17.2.4 spesifikasyonuna göre DR7 kaydı kodlanır. Her slot için yerel etkinleştirme (L biti), koşul (R/W bitleri) ve uzunluk (LEN bitleri) ayrı ayrı ayarlanır. Execute tipinde donanım kesme noktası, *EXCEPTION\_SINGLE\_STEP* istisnasını üretir; DR6 kaydı hangi slorun tetiklendiğini bildirir.

### 2) PAGE\_GUARD Mekanizması

*VirtualProtectEx()* ile hedef bellek bölgesine *PAGE\_GUARD* bayrağı eklenir. Paketleyicinin bu bölgeye ilk erişiminde *STATUS\_GUARD\_PAGE\_VIOLATION* istisnası üretilir; Windows bayrağı otomatik kaldırır. Erişim adresi OEP adayı olarak kaydedilir ve tüm iş parçacıkları *SuspendThread()* ile askıya alınır.

**Tablo II. DebuggerEngine İstisna İşleyici Akışı**

| İstisna Kodu                | İşlem                                                   |
|-----------------------------|---------------------------------------------------------|
| EXCEPTION_BREAKPOINT        | İlk INT3 (yükleyici) yutulur; sonrakiler yazılım BP'dir |
| EXCEPTION_SINGLE_STEP       | DR6 okunur; Execute BP ise OEP kaydedilir               |
| STATUS_GUARD_PAGE_VIOLATION | Hata adresi OEP adayıdır; tüm thread'ler askıya alınır  |
| EXCEPTION_ACCESS_VIOLATION  | İşlenmeden debuggee'ye iletilir                         |
| Diğerleri                   | DBG_EXCEPTION_NOT_HANDLED döndürülür                    |

## C. Dumper

Bellek dökümünün temel zorluğu, disk ve bellek hizalama değerlerinin farklı olmasıdır: *disk*  $\square$  *FileAlignment* (tipik 0x200) ile, bellekte ise *SectionAlignment* (tipik 0x1000) ile hizalanır. Dumper bu sorunu 'memory-aligned dump' yaklaşımıyla çözer:

- *FileAlignment*  $\leftarrow$  *SectionAlignment* olarak ayarlanır.
- *PointerToRawData[i]*  $\leftarrow$  *VirtualAddress[i]*
- *SizeOfRawData[i]*  $\leftarrow$  *align(VirtualSize[i], FileAlignment)*

Bu yöntem Scylla'nın varsayılan davranışıyla örtüşmekte ve PE spesifikasyonu çerçevesinde tam geçerlilik taşımaktadır. Hem 32-bit hem 64-bit PE formatı desteklenmekte; OptionalHeader magic değerinden mimari otomatik algılanmaktadır.

## D. IATRebuilder

Paketlenmiş yürütülebilirler, yeniden oluşturulan döküm dosyasında özgün ithalat tablosunu (IAT — Import Address Table) içermeyebilir. IATRebuilder, Scylla tarzı bir algoritmayla bu tabloyu dinamik olarak yeniden inşa eder:

**Adım 1-2:** *EnumProcessModulesEx()* ile yüklü modüller keşfedilir; her modülün export tablosu *ReadProcessMemory()* üzerinden okunarak *adres*  $\rightarrow$  (*modül*, *fonksiyon*) haritası oluşturulur. Bu adımda *LoadLibrary()* kullanılmaz; hedef ve araç sürecinin adres uzayları ayrıdır.

**Adım 3-4:** Belirtilen IAT bölgesi işaretçi işaretçi taranır. Geçerli bir export adresine işaret eden her giriş bir 'chunk' (tek DLL'e ait ardışık importlar grubu) olarak sınıflandırılır.

**Adım 5-6:** *IMAGE\_IMPORT\_DESCRIPTOR* dizisi, INT (Import Name Table), *IMAGE\_IMPORT\_BY\_NAME* blokları ve DLL adı stringleri tek bir ikili blob içinde derlenir; '.scy0' adlı yeni bir bölüm olarak Dumper'a enjekte edilir ve *DataDirectory[IMPORT]* güncellenir.

## IV. GUI IMPLEMENTASYONU

Araç, Win32 yerel API'si kullanılarak geliştirilmiş grafik kullanıcı arayüzüyle de sunulmaktadır. *WinMain* giriş noktasına sahip *unpacker\_gui* hedefi, konsol penceresi açmadan doğrudan arayüzü gösterir.

### A. Mimari: WndProc Yönlendirici

Win32 pencereleri *GWLP\_USERDATA* alanında *MainWindow* nesnesinin adresini saklar. Statik *WndProcStatic()* yönlendirici bu adresi alarak üye metot *WndProc()*'a iletir; böylece nesne yönelimli bir Win32 sarmalayıcı elde edilmiş olur.

### B. Thread Güvenliği

Unpack boru hattı ayrı bir worker iş parçacığında çalışır; UI iş parçacığı donmaz. Motor logları *Logger::SetSink()* aracılığıyla yakalanır ve *PostMessage(WM\_APP\_LOG)* ile UI iş parçacığına gönderilir; bu yaklaşım Win32'nin iş parçacığı güvenliği gerekliliklerini karşılar.

### C. Kontroller ve Kullanıcı Deneyimi

Arayüz; hedef / çıktı EXE seçiciler, HW Execute BP adres giriş alanı, IAT yeniden inşa VA/boyut alanları, PE Analizi / UNPACK BAŞLAT düğmeleri ve çok satırlı salt okunur bir log alanından oluşur. Dosya seçici açıldığında çıktı yolu otomatik önerilmektedir.

## V. DERLEME VE KULLANIM

### A. Gereksinimler

- Windows 10/11 (64-bit)
- MSVC 2022 (C++20 desteği, /W4 /permissive-)
- CMake  $\geq 3.20$
- Bağlı kütüphaneler: psapi, comctl32, comdlg32, shell32

### B. CMake Yapılandırması

Proje iki hedef tanımlar: **unpacker** (CLI) ve **unpacker\_gui** (Win32 GUI). Ortak motor kaynak dosyaları (PEParser.cpp, ProcessManager.cpp, DebuggerEngine.cpp, Dumper.cpp, IATRebuilder.cpp) ENGINE\_SOURCES

değişkeninde gruplanarak her iki hedef tarafından paylaşılır.

### C. CLI Kullanım Örneği

```
unpacker.exe ornek.exe cikarilmis.exe ^
--bp-exec 0x7FF71B9619C0 ^
--iat 0x7FF71B970000 0x400
```

Sayısal değerler ondalık veya 0x önekli onaltılık olarak verilebilir. --bp-exec seçeneği donanım Execute kesme noktası kurar; --iat seçeneği IAT yeniden inşasını etkinleştirir.

### D. Hata Kodları

Tablo III. Çıkış Kodu Anlamları

| Kod | Anlam                    |
|-----|--------------------------|
| 0   | Başarı                   |
| 2   | OEP tespit edilemedi     |
| 3   | OEP ImageBase'in altında |
| 10  | PE ayrıştırma hatası     |
| 11  | Süreç yönetimi hatası    |
| 12  | Bellek dökümü hatası     |
| 13  | IAT yeniden inşa hatası  |
| 99  | Beklenmedik istisna      |

## VI. SONUÇ

Bu çalışmada, paketlenmiş Windows PE yürütülebilir dosyalarını çalışma zamanında çözmeye yönelik nesne yönelimli bir araç tasarlanmış ve C++20 ile implement edilmiştir. Sistem; modüler mimari, SOLID tasarım ilkeleri ve RAIL idiomunu merkeze alarak geliştirilmiştir.

DebuggerEngine, donanım kesme noktaları ve PAGE\_GUARD mekanizmalarını birleştirerek güvenilir OEP tespiti sağlamaktadır. Dumper, bellek hizalaması farkını şeffaf biçimde ele alırken IATRebuilder, Scylla benzeri bir algoritmayla ithalat tablosunu çalışma

zamanı bilgisinden yeniden oluşturmaktadır. GUI bileşeni, worker iş parçacığı ve PostMessage tabanlı günlük iletişimi sayesinde UI donması yaşanmadan kullanılabilir.

Olası geliştirmeler arasında; PEParse ve ProcessManager sınıfları için ayrı kaynak dosyaları oluşturulması, emülatör tabanlı bir IUnpacker stratejisi eklenmesi, kod örtüşme tabanlı OEP doğrulama ve çok katmanlı paketleyici desteği sayılabilir.

## VII. TEŞEKKÜR

Bu proje, Nesne Yönelimli Programlama dersi kapsamında gerçekleştirilmiştir. Ders materyalleri ve yönlendirmeleri için danışman hocama teşekkürlerimi sunarım.

## VIII. KAYNAKLAR

- [1] Microsoft Corporation, "Debug Event Handling," Windows Dev Docs, <https://learn.microsoft.com/en-us/windows/win32/debug/debug-events>, 2024.
- [2] Intel Corporation, "Intel® 64 and IA-32 Architectures Software Developer's Manual," Vol. 3, Chap. 17, 2024.
- [3] NtCore, "Scylla Import Reconstructor," <https://github.com/NtQuery/Scylla>, 2024.
- [4] ISO/IEC, "Programming Languages — C++, ISO/IEC 14882:2020 (C++20), 2020.
- [5] B. Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.
- [6] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.
- [7] Microsoft Corporation, "Portable Executable (PE) Format Specification," <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>, 2024.